

Software Maintenance

การบำรุงรักษาซอฟต์แวร์และกระบวนการพัฒนาอย่างยั่งยืน

รายชื่อสมาชิกกลุ่ม

1. อรปรียา อาสนทอง (6605001699)
2. อภิญญา วงษ์ศรี (6605005344)
3. กิมทีนา ลี (6705006069)
4. สุธิดา ทรงรัมย์ (6605005716)
5. วิญญู สายแสง (6805007405)



1. ความหมายของการบำรุงรักษาซอฟต์แวร์

Software Maintenance คือ การปรับเปลี่ยนซอฟต์แวร์หลังการส่งมอบ เพื่อให้ซอฟต์แวร์ยังคงใช้งานได้ตามความต้องการและเหมาะสมกับสภาพแวดล้อมที่เปลี่ยนแปลง

อ้างอิงตามมาตรฐาน **ISO/IEC/IEEE 14764:2022** ที่ระบุแนวทางและประเภทของการบำรุงรักษาอย่างชัดเจน

สรุปง่าย ๆ: ไม่ได้หมายถึงการแก้ Bug อย่างเดียว แต่ครอบคลุมการแก้ไข ปรับตัว ปรับปรุง และป้องกันปัญหาตลอดอายุการใช้งาน





| เหตุผลที่ต้องบำรุงรักษาซอฟต์แวร์

- พบข้อผิดพลาดหลังจากนำระบบไปใช้งานจริง
- ระบบปฏิบัติการ Hardware, Database หรือ API ภายนอกเปลี่ยนแปลง
- ผู้ใช้หรือองค์กรต้องการความสามารถใหม่ๆ เพิ่มขึ้น
- ต้องการเพิ่มประสิทธิภาพหรือความสามารถในการบำรุงรักษา (Maintainability)
- ลดความเสี่ยงจากข้อผิดพลาดที่ยังไม่เกิดขึ้น
- ข้อกำหนดทางธุรกิจหรือสภาพแวดล้อมของระบบเปลี่ยนแปลง

ควรมีการวางแผนตั้งแต่ช่วงพัฒนาซอฟต์แวร์ ไม่ใช่รอจนระบบมีปัญหา



2. ประเภทของการบำรุงรักษา (ISO/IEC/IEEE 14764)



1. Corrective

แก้ไขซอฟต์แวร์หลังจากพบปัญหาหรือข้อผิดพลาด เพื่อให้ระบบกลับมาทำงานถูกต้อง (Reactive)



2. Adaptive

ปรับเปลี่ยนซอฟต์แวร์เพื่อให้ทำงานได้เมื่อสภาพแวดล้อมเปลี่ยนไป เช่น OS, Database, API หรือ Hardware



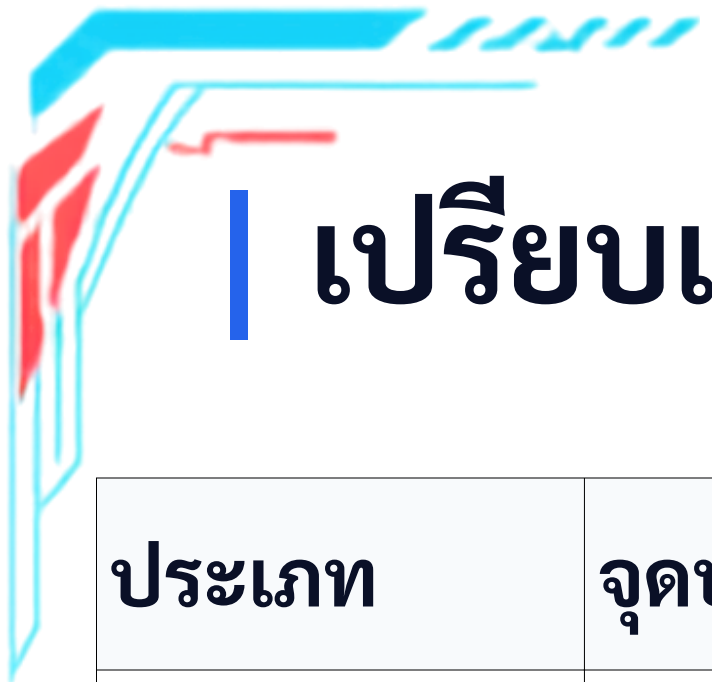
3. Perfective

ปรับปรุงหลังส่งมอบ เพื่อเพิ่มความสามารถ ประสิทธิภาพ หรือคุณภาพของซอฟต์แวร์ให้ดียิ่งขึ้น



4. Preventive

ค้นหาและแก้ไขปัญหาล่วงก่อนกลายเป็นข้อผิดพลาดจริง เป็นแบบ Proactive เช่น Refactoring



| เปรียบเทียบประเภทของ Software Maintenance

ประเภท	จุดประสงค์หลัก	ตัวอย่างงาน
Corrective	แก้ปัญหาที่พบแล้วหลังใช้งานจริง	แก้ Bug คำนวณราคาสินค้าหรือภาษีผิดพลาด
Adaptive	ปรับให้เข้ากับ Environment ใหม่	รองรับการอัปเดต Database Server หรือ OS รุ่นใหม่
Perfective	เพิ่มความสามารถหรือปรับปรุงคุณภาพ	เพิ่มระบบค้นหาข้อมูล (Search) และปรับ Performance
Preventive	ป้องกันปัญหาที่อาจเกิดขึ้นในอนาคต	ทำ Refactoring Code ที่มีความซับซ้อนสูง

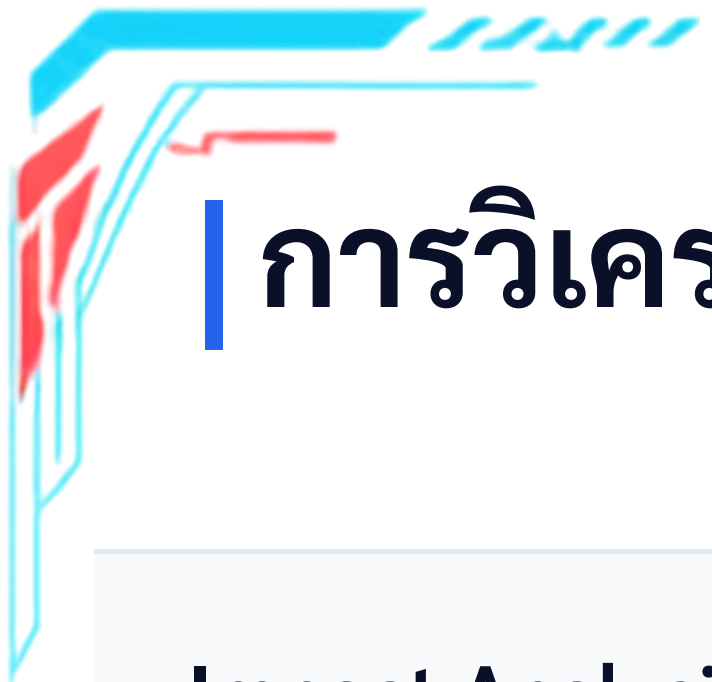


3. กระบวนการบำรุงรักษาซอฟต์แวร์

การบำรุงรักษาควรดำเนินการอย่างเป็นกระบวนการผ่านขั้นตอนที่ควบคุมได้ เพื่อป้องกันความผิดพลาดลูกกลาม

1. Request / Problem Report
2. Identification & Classification
3. Impact Analysis
4. Planning & Prioritization
5. Design & Implementation
6. Testing & Regression Testing
7. Review / Approval
8. Release / Deployment
9. Evaluation & Closure

ทุกคำขอควรถูกบันทึกเป็น **Modification Request (MR)** เพื่อตรวจสอบย้อนกลับได้



| การวิเคราะห์ผลกระทบและการทดสอบซ้ำ

Impact Analysis

วิเคราะห์ว่าการเปลี่ยนแปลงจะส่งผลกระทบต่อส่วนใดบ้าง เช่น Source Code, Database, API, Module อื่น, Test Case และ Documentation

ตัวอย่าง: การเปลี่ยนโครงสร้างข้อมูล Customer อาจกระทบ Order, Payment และ Report

Regression Testing

การทดสอบซ้ำเพื่อยืนยันว่าการแก้ไขหรือเพิ่มฟังก์ชันใหม่ **ไม่ได้ทำให้ฟังก์ชันเดิมที่เคยทำงานได้ปกติเกิดความเสียหาย**

มีความสำคัญมากในระบบที่มีการอัปเดตและบำรุงรักษาต่อเนื่อง



4. เทคนิคการบำรุงรักษา (ตอนที่ 1)

Refactoring

ปรับโครงสร้างภายใน Source Code โดยไม่เปลี่ยน
พฤติกรรมภายนอก ช่วยลด Code Complexity และ
Duplication

```
Function ใหญ่รวมทุกอย่าง  
↓ (Refactoring)  
แยกเป็น: validateData(), calculate(), saveData(), sendEmail()
```

Reverse Engineering

วิเคราะห์ซอฟต์แวร์ที่มีอยู่เพื่อทำความเข้าใจโครงสร้าง
และการออกแบบ เหมาะกับระบบเก่าที่ไม่มีเอกสาร

```
Existing System → Analyze Code → Understand Architecture → Create  
Documentation
```




| เทคนิคการบำรุงรักษา (ตอนที่ 2)



Program Comprehension

ทำความเข้าใจโปรแกรมผ่าน
Code Reading, Call Graph,
UML Diagram และ
Debugging Tools ก่อนลงมือ
แก้ไข



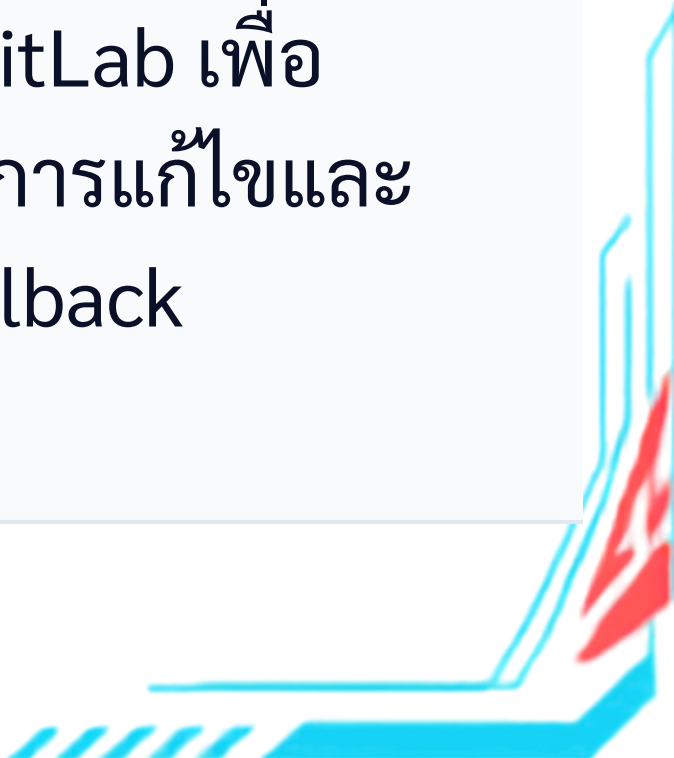
Static Analysis

ใช้เครื่องมือตรวจสอบ Code
Smell, Duplicate Code และ
Security Issue โดยไม่ต้องรัน
โปรแกรม



Version Control

ควบคุม Version ด้วย Git,
GitHub หรือ GitLab เพื่อ
ติดตามประวัติการแก้ไขและ
รองรับการ Rollback



สรุปสังท้าย: Software Maintenance เป็นหัวใจสำคัญของ
วงจรชีวิตซอฟต์แวร์

การวางแผนที่ดี กระบวนการที่รัดกุม และเทคนิคที่เหมาะสม จะช่วยลด
ต้นทุนและยกระดับคุณภาพซอฟต์แวร์ในระยะยาว

